Data Analytics | Group projects

Expected Goals Model

By: Kaspar Soukup, Guy Kaye, Pengcheng Xu, Mohan Lou, Elisa Montange

Abstract

Small moments in football can completely alter the course of a match. One wrong pass by a defender can lead to a lucky goal for the underdog, who goes on to win the whole game. As football is such a low-scoring sport, it is highly vulnerable to luck. The final score does not always tell the full story, and the better team does not always win. Therefore, obtaining a measure that evaluates the quality of chances a team creates, can provide a much better indicator of who deserved to win the match. **Expected Goals (xG)** helps bridge this gap by **estimating the probability that a shot will result in a goal based solely on the difficulty of the chance at the moment it is taken**. As the name implies, it measures how many goals a team *should have* scored based on the quality of the shots they created.

Building a robust Expected Goals model with reliable probabilities requires collecting extensive historical data from many shots with various conditions. A shot from close range can be easy when there are no defenders around, or extremely difficult if taken from a tight angle under pressure. With the right data and models, we can measure the probability of all these shots and credit teams for creating high-potential opportunities regardless of the outcome.

Data Acquisition

We acquired the data for our analysis through StatsBomb's public API, accessed via their official Python package **statsbombpy**. StatsBomb is one of the leading providers of football event data worldwide, supplying detailed on-ball event-level datasets to professional clubs and analysts. Their open-data release provides free access to several competitions across multiple years, ideal to collect a large amount of shot data for our model

Before settling on StatsBomb, we also considered web-scraping data from Understat, a website that provides publicly available shot-level data, its own xG model, and game statistics. However, we ultimately chose StatsBomb because it offers richer, more granular event-level detail, such as pressure information and rich location data, which are highly relevant factors for expected goal models.[1]

We obtained the dataframe of all shot events from the **2015/2016 season** across the top five European leagues: the English Premier League, Spanish La Liga, German Bundesliga, Italian Serie A, and French Ligue 1. This dataset consisted of **over 45,000 shots**, with each row representing a unique shot taken in a match.

Exploratory Data Analysis and Data Preprocessing

The raw dataset included 40 variables, some being unrelated to shot events. We therefore selected only features relevant for predicting shot success, guided by football intuition and prior research on xG models. These features include event metadata, shot technique, situation, location, and outcome (see appendix A for details).

We relied on the **StatsBomb Data Specification v1.1**, which specifies how each event and sub event is encoded. This was particularly important because some fields labelled as null or NA do not represent missing data but instead indicate a False value for a given qualifier, for example "under pressure".

We removed penalty kicks and corner shots because these situations differ substantially from regular open play shots. The dataset contained fewer than 500 penalties and only 13 corner attempts out of more than 45,000 total shots. Excluding them still left us with ample data to train our models.

Next, we encoded the categorical variables using one-hot encoding, except for the "*body_part*" variable, where left and right foot were combined due to the absence of dominant-foot information.

[1] (David, Arslan and Robberechts, 2020)

We then binary encoded (1 = goal, 0 = no goal) the dependent variable. The resulting distribution exhibited a pronounced class imbalance, with a goal conversion rate of approximately 10%. Further inspection revealed unusually high success rates within the "Other" categories for both the "play_pattern" and "body_part" features. For *"body_part"* the documentation specifies that "Other" included less common contact areas such as chest and knee. We hypothesized that the higher success rate was a result of proximity, assuming players utilize these parts only when already close to the goal. This hypothesis was supported by visualizing the average shot distance per categorical variable. Similarly, the "Other" category for "*play_pattern*" exhibited a lower than average distance to the goal. However, the available documentation provided no further explanation regarding the contents of this specific category. In line with expectations both "*shot_open_goal*" and "*shot_one_on_one*" show higher success rates (0.8 and 0.25 respectively). See appendix B for visualizations.

We also examined shot to goal ratios across teams. Clubs such as Real Madrid and Barcelona required far fewer attempts to score (about 7 and 6 shots per goal) than teams such as Aston Villa, which needed more than 15 (see appendix C).

Finally, visualising the spatial distribution of shots showed a strong tendency for attempts to be taken from the centre of the penalty area (see appendix D). Although this pattern is expected, our model will allow us to quantify the likelihood of scoring from these and other locations.

Feature Engineering

Although the raw x-y coordinates for each shot attempt were available, established literature on Expected Goal (xG) modeling consistently indicates that transforming these into spatial geometric features yields more robust and predictive models.[2] Therefore, we engineered several key spatial features: the Euclidean distance from the shooter to the center of the goal, the shot angle, the opposition goalkeeper's position, and the distance between the shooter and the goalkeeper (see appendix E). Facilitating these geometric calculations was the data standardization implemented by StatsBomb, as detailed in their documentation. The shot coordinates are consistently mapped to the attacking half of the pitch (specifically, the right half), regardless of the actual side of the field the shot was taken on (refer to appendix F for the coordinate system definition).

Modelling

We model expected goals (xG) as a binary classification task, with goals coded as 1 and non-goals as 0. The data were randomly split into 75% training and 25% testing sets using a fixed random seed. All models were tuned on the training data using a consistent cross-validation protocol and evaluated on the same held-out test set. Because goals account for only about 10% of observations, model performance is assessed using AUC-ROC to evaluate ranking ability and calibration metrics, including the Brier score, to assess the quality of predicted probabilities. Accuracy is not reported, as a trivial classifier predicting no goal for every shot would achieve roughly 90% accuracy while offering no meaningful probabilistic insight. Calibration is therefore central, as an xG model must produce probabilities that reflect true scoring likelihoods.

Logistic Regression (L1 Regularisation)

Logistic regression with L1 regularisation serves as a linear benchmark. The L1 penalty was chosen to address multicollinearity and perform implicit feature selection in a high-dimensional setting. Categorical variables were one-hot encoded with baseline categories removed, correlated predictors were excluded, and continuous variables were standardised prior to estimation. The regularisation strength was selected

[2] (Davis and Robberechts, 2020), (Hewitt and Karakuş, 2023), (Rathke, 2017)

to maximise AUC-ROC. The optimal specification achieved a cross-validated AUC of 0.806 and a test AUC-ROC of 0.795, providing strong baseline discrimination. As expected, recall for goals was low under default classification thresholds, highlighting the limits of linear decision boundaries in capturing subtle shot quality differences. Nonetheless, the model offers transparency, interpretability, and a useful reference point for more complex approaches.

Boosting Models (AdaBoost, XGBoost, CatBoost)

Boosting-based models were evaluated to better capture complex shot patterns while maintaining probabilistic interpretability. AdaBoost selected a relatively expressive ensemble and achieved a test AUC-ROC of 0.798, improving upon the linear benchmark but producing probabilities that were heavily concentrated at low values. This compression limited its usefulness for differentiating among high-probability chances, which is a key requirement in xG applications. XGBoost further improved discrimination, reaching a test AUC-ROC of 0.801 by combining shallow trees with a large ensemble and explicit class imbalance adjustment. In this dataset, the preference for shallow trees suggests that a goal is driven by many small, additive effects rather than deep hierarchical splits. While XGBoost better separated high-quality chances, its predicted probabilities remained conservative at the upper tail, indicating mild underconfidence for the most dangerous shots. CatBoost delivered the strongest overall performance among boosting methods, achieving a test AUC-ROC of 0.806 and the lowest Brier score (0.072). Its probability estimates were more stable across the full range of shot qualities, making it particularly well-suited to this study where reliable probability calibration is as important as ranking ability.

Neural Network

The neural network achieved competitive discrimination, with performance comparable to the strongest boosting models. In this study, model selection favored a feedforward architecture with a single hidden layer of 128 neurons and ReLU activation, which provided sufficient flexibility to rank shots effectively while avoiding overfitting; deeper architectures yielded no additional gains. While ranking performance was strong, predicted probabilities exhibited mild underconfidence at higher values, requiring post-hoc calibration to better align predicted and observed scoring rates. Given the limited improvement in both discrimination and calibration relative to boosting methods, the neural network did not offer a clear advantage for xG estimation in this application, despite its greater modeling flexibility.

Random Forest

The Random Forest model produced robust and stable performance, achieving a test AUC-ROC of 0.802 and a Brier score of 0.072, indicating well-calibrated probability estimates on average. In this study, the best-performing specification used 400 trees with a maximum depth of 10, a minimum split size of 20, and $\log_2$ feature subsampling, which effectively smoothed over noise in individual shot features and yielded reliable mid-range probability estimates. However, like other ensemble methods, the model struggled to identify rare goal events under default thresholds and offered no improvement in separating the highest-quality chances compared to gradient boosting. As a result, while Random Forests provided strong baseline calibration and discrimination, they were ultimately less effective than boosting methods for fine-grained xG estimation.

Results

From the Evaluation table (appendix G), the strongest-performing models are CatBoost, Random Forest, and XGBoost in terms of overall discriminative performance, as measured by ROC AUC. CatBoost achieves the highest AUC (0.806) and the lowest Brier score (0.072), indicating the best balance between ranking ability and probability calibration among all candidates. Random Forest and XGBoost

follow closely, with comparable AUC values (0.802 and 0.801, respectively) and similarly strong calibration performance (see appendix G-K)

Across all models, weighted recall and precision remain high due to the dominance of non-goal events, while recall for goals is uniformly low, reflecting the severe class imbalance inherent in goal data. This underscores the limited informativeness of threshold-dependent metrics in this setting and reinforces the importance of ROC AUC and calibration measures for xG modeling. Consistent with the ranking results, CatBoost, Random Forest, and XGBoost demonstrate superior ability to distinguish between high- and low-quality scoring chances, while also producing probability estimates that more faithfully reflect observed goal frequencies.

Every model is useless without a clear application for it. We, thus, developed a prototype that enables managers and trainers to get an estimate for the expected goals of their teams. They can input the location and describe the shot through variables like technique, body part used or play pattern and get an expected goals value in return. The prototype can be accessed here: https://xgoals.streamlit.app/.

Feature importance

The feature importance analysis for the Catboost model, visualized in appendix L, reveals critical insights into the factors influencing shot outcomes. The “shot_angle” emerges as the most significant predictor, because a wider angle in the pitch directly correlates with a higher probability of scoring. Following this, key spatial and positional features such as “distance_player_gk”(distance between the player and the goalkeeper), “distance_from_goal_left_post”, “distance_from_goal_right_post”, and the “goalkeeper_x” position are identified as highly influential. These features quantify the immediate geometric context of the shot, providing direct information on the shot’s quality and the defensive stance. Least impactful features are various play patterns and specific shot techniques. Though they contribute to the shot’s context, their influence is much less powerful than fundamental spatial attributes. The model prioritizes features that offer direct quantification of the shot’s physical traits, instead of shooters’ characteristics.

Summary

Our study shows that spatial geometry is far more influential in predicting goals than contextual or stylistic features. Although CatBoost achieved the best overall performance, differences across models were modest, suggesting that information contained in the data itself matters. This limited performance gap likely reflects constraints in the dataset, including sparse goal events and missing player and pressure related information. The final model is deployed as a prototype tool for practical coaching and analysis, while future work should focus on incorporating richer contextual and player-specific data to further improve predictive accuracy.

Possible Improvements and future research

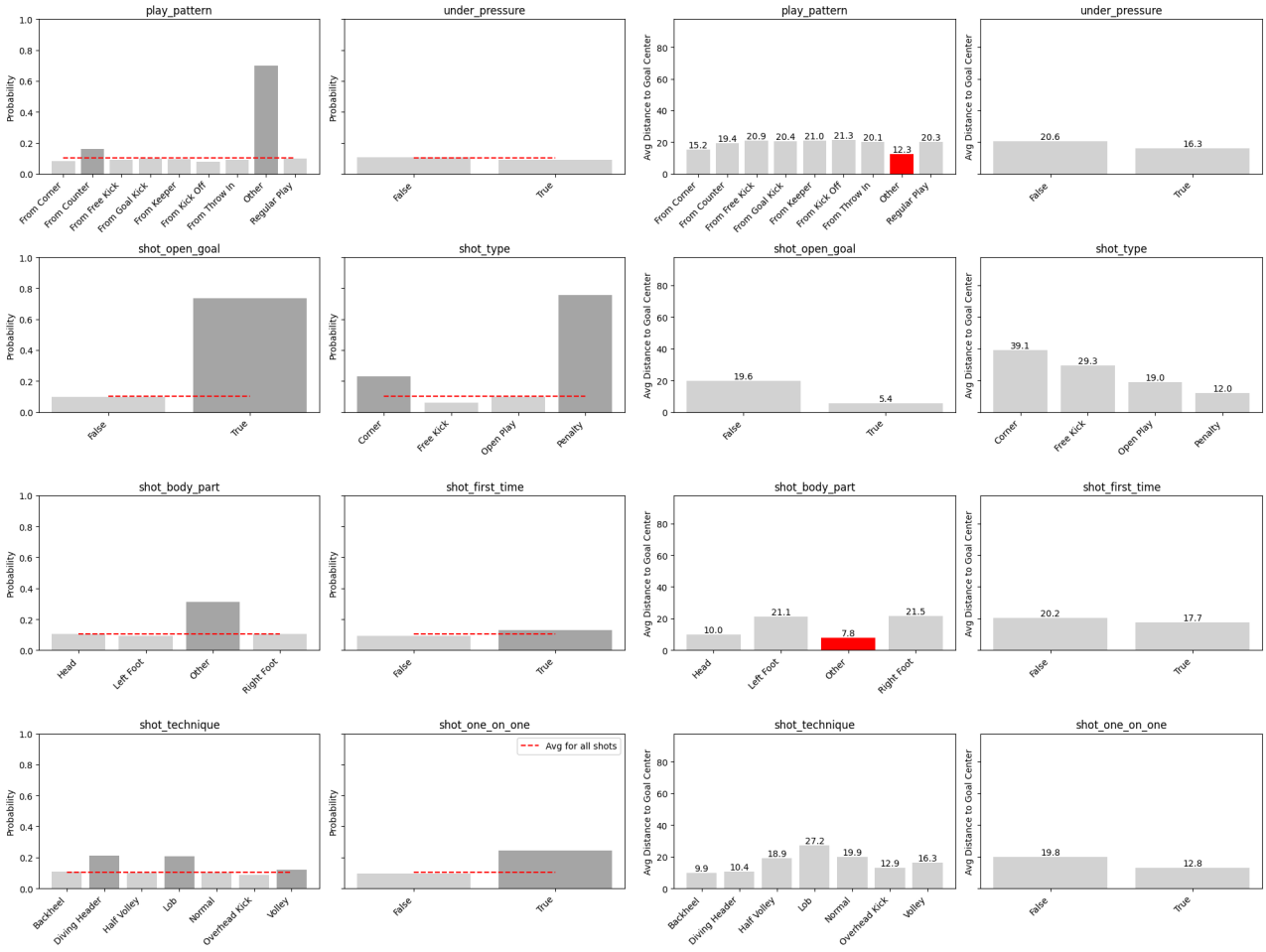
Future improvements could enhance the model by incorporating richer player- and context-specific data, such as dominant foot and the positions of defenders and teammates, enabling a more accurate representation of pressure and one-on-one situations beyond existing event tags. Extending the framework to player- and team-specific xG models would further capture differences in finishing ability and chance creation, while analyzing how xG profiles translate across leagues or change when players join stronger teams could provide valuable insights for recruitment, valuation, and performance forecasting.

Appendix

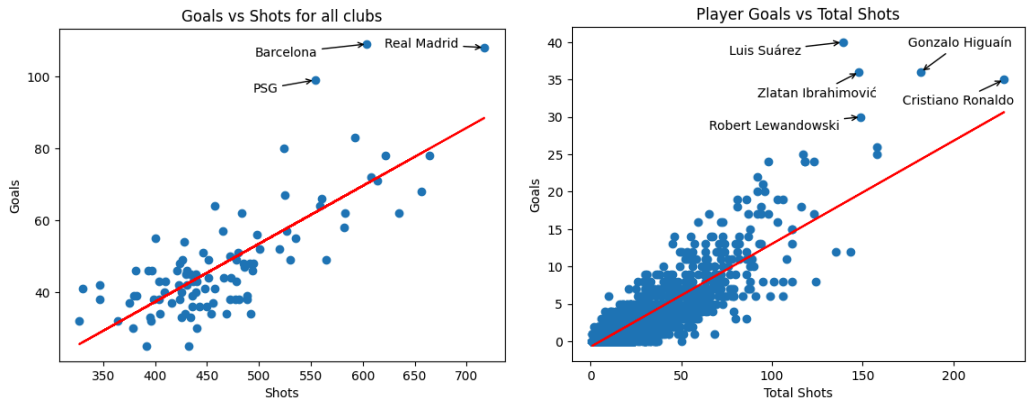
A. Feature overview

Column Name	Type	Description
player_x, player_y	Numeric	X-coordinate and Y-coordinate of the player taking the shot
goalkeeper_x, goalkeeper_y	Numeric	X-coordinate and Y-coordinate of the opposition goalkeeper
distance_from_goal_* (center, left_post, right_post)	Numeric	Distance from the shooter to the center of the goal
gk_distance_from_goal_* (center, left_post, right_post)	Numeric	Distance from the goalkeeper to the center of the goal
distance_player_gk	Numeric	Euclidean distance between the shooter and the goalkeeper
shot_angle	Numeric	The angle of the shot relative to the goal posts (in radians)
shot_open_goal	Binary (0/1)	Indicates if the goal was open (1 = True, 0 = False)
under_pressure	Binary (0/1)	Indicates if the shooter was under pressure (1 = True, 0 = False)
shot_one_on_one	Binary (0/1)	Indicates if it was a one-on-one situation (1 = True, 0 = False)
shot_first_time	Binary (0/1)	Indicates if the shot was taken first-time (1 = True, 0 = False)
body_part_* (foot, head, other)	Binary (0/1)	A set of One-Hot encoded columns representing the body part used for the shot
shot_technique_* (normal, volley, lob, …)	Binary (0/1)	A set of One-Hot encoded columns representing the technique
play_pattern_* (regular_play, counter, …)	Binary (0/1)	A set of One-Hot encoded columns representing the play pattern
shot_type_* (open_play, free_kick)	Binary (0/1)	A set of One-Hot encoded columns representing the shot type

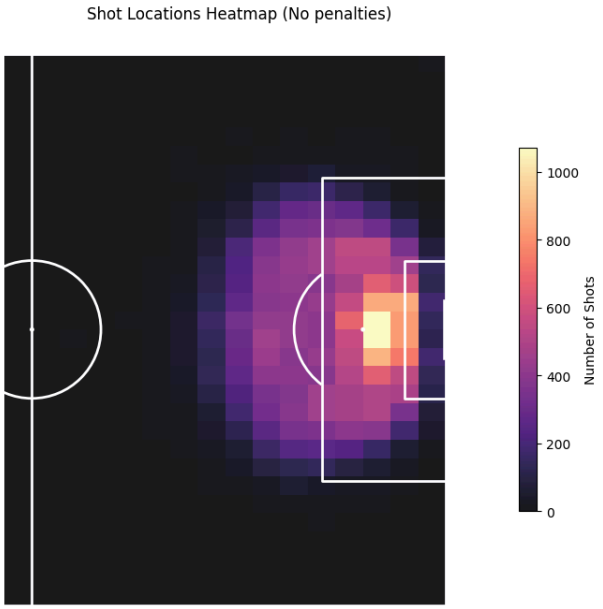
B. Shot success rates and average distance to the goal for predictor variables



C. Simple prediction for goals scored based on shots taken



D. Heatmap of shots taken (excluding penalties)

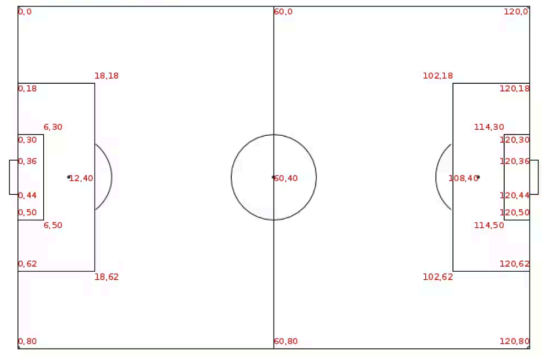


E. Spatial features

Feature Name	Formula / Logic
player_x, player_y	$x = \text{location}[0], \quad y = \text{location}[1]$
goalkeeper_x, goalkeeper_y	$x_{gk} = \text{freeze\_frame}[0], \quad y_{gk} = \text{freeze\_frame}[1]$
distance_from_goal_ (center, left_post, right_post)	$\sqrt{(120 - x)^2 + (Y_{target} - y)^2}$ *where* $Y_{target} \in \{40, 36, 44\}$
gk_distance_from_goal_ (center, left_post, right_post)	$\sqrt{(120 - x_{gk})^2 + (Y_{target} - y_{gk})^2}$ *where* $Y_{target} \in \{40, 36, 44\}$
distance_player_gk	$\sqrt{(x - x_{gk})^2 + (y - y_{gk})^2}$
shot_angle	$\lvert\arctan(44 - y, 120 - x) - \arctan(36 - y, 120 - x)\rvert$

F. Statsbomb coordinate system

Pitch Coordinates - Coordinates specified as (x, y).



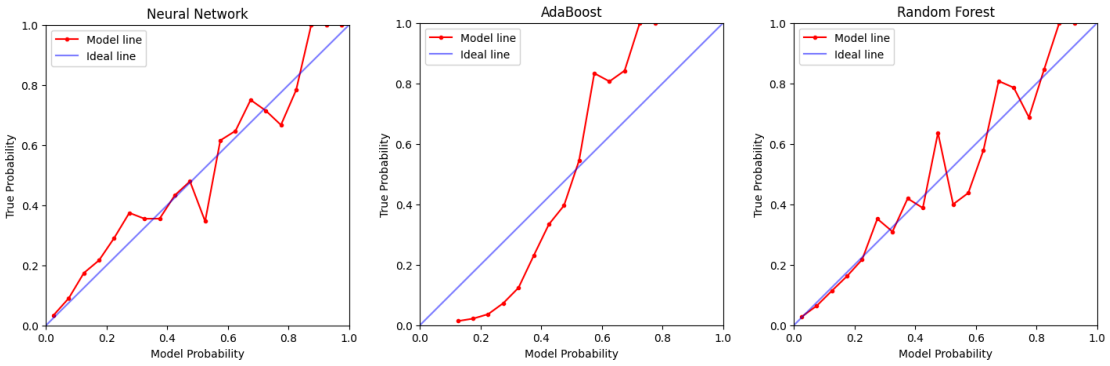
G. Evaluation Table

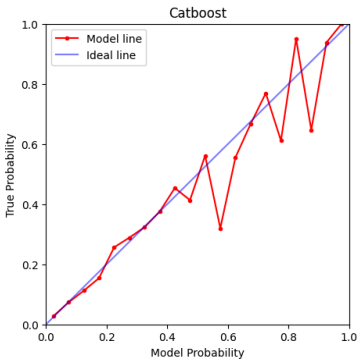
	Recall (weighted)	Recall (Goal)	Precision (weighted)	Precision (Goal)	F1-score (weighted)	ROC AUC	Brier score
Lasso Regression	0.91	0.12	0.89	0.61	0.88	0.7953	0.0732
Random Forest	0.91	0.08	0.89	0.67	0.87	0.8020	0.0724
AdaBoost	0.91	0.10	0.89	0.71	0.88	0.7984	0.1030
XGBoost	0.91	0.10	0.89	0.69	0.88	0.8035	0.0720
Catboost	0.91	0.12	0.89	0.65	0.88	0.8055	0.0719
Neural Network	0.91	0.10	0.89	0.69	0.88	0.7993	0.0731

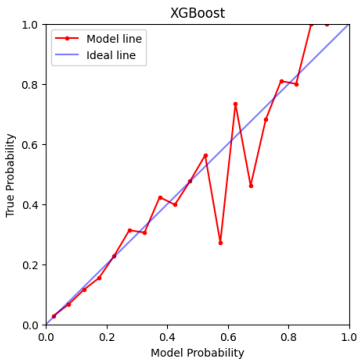
H. Brier Scores (lower is better)

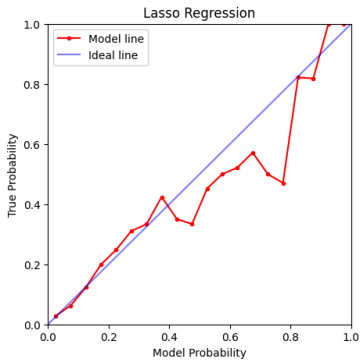
Catboost	0.0719
XGBoost	0.0720
Random Forest	0.0724
Neural Network	0.0731
Lasso Logistic Regression	0.0732
Adaboost	0.1030

I. Calibration Curves

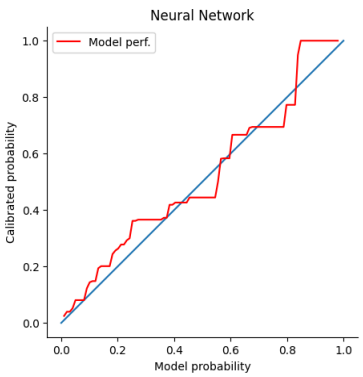


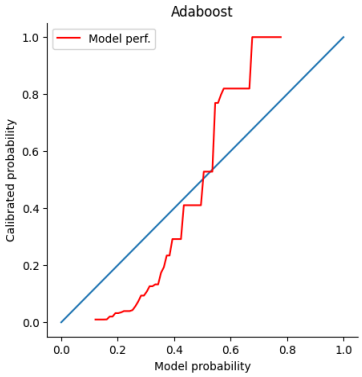


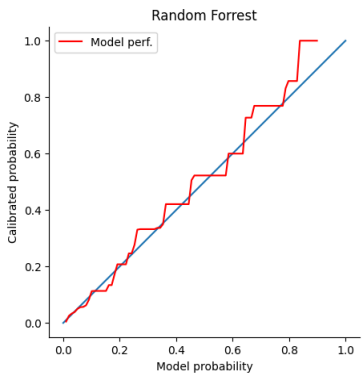


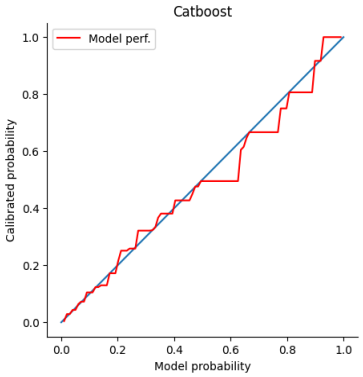


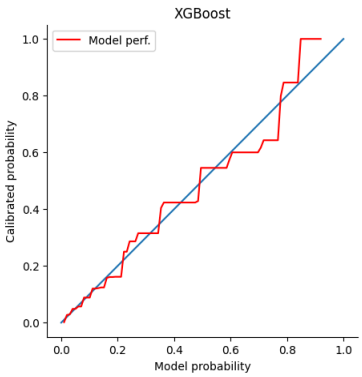
J. Isometric Calibration Curves

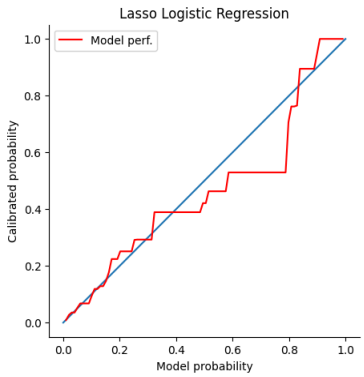


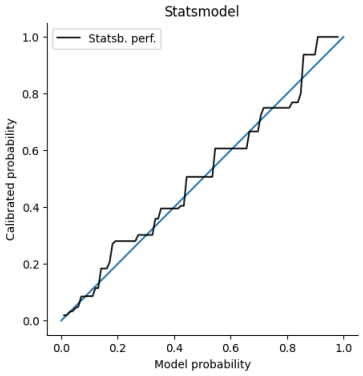








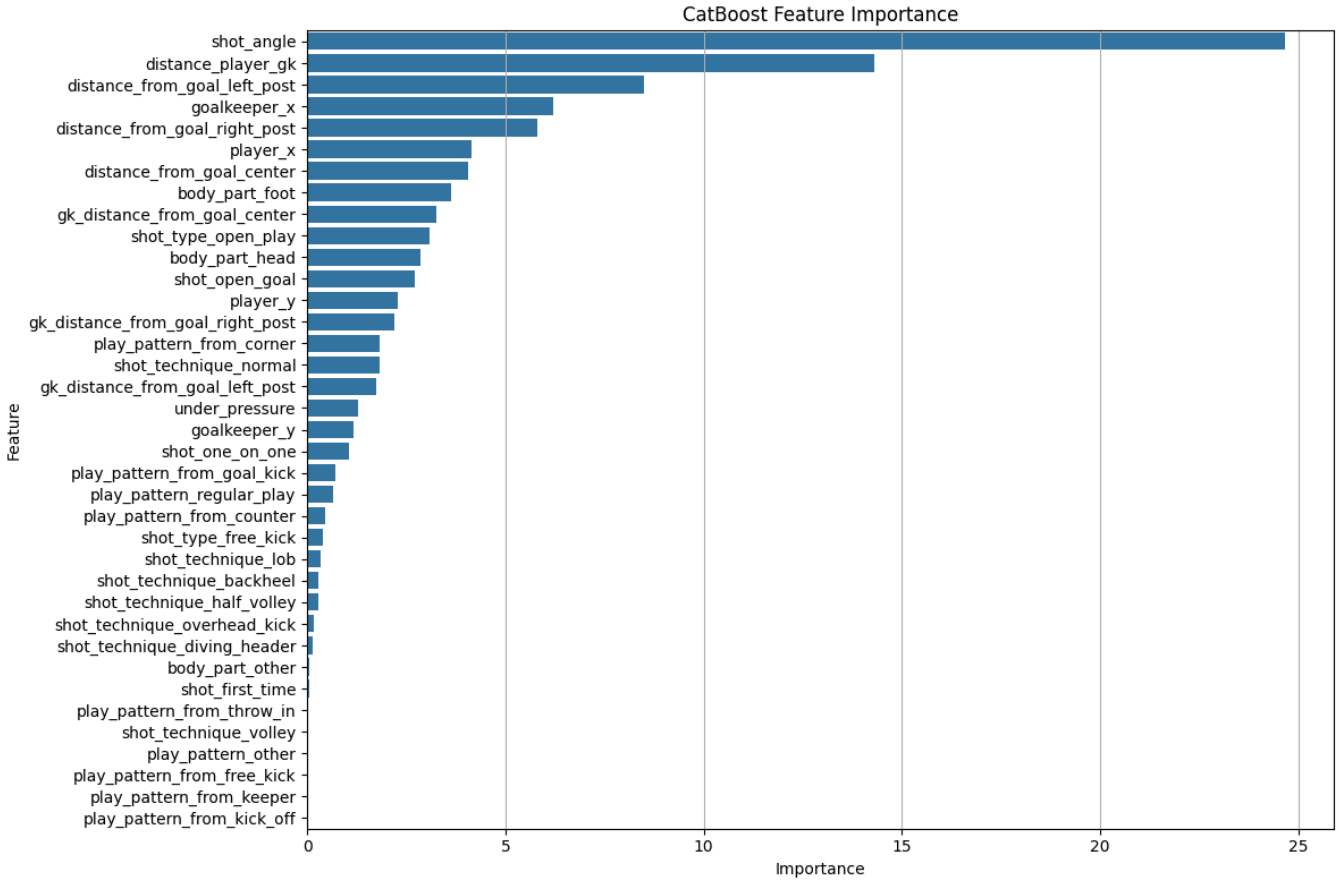




K. AUC ROC

XGBoost	0.804
Random Forest	0.802
Neural Network	0.799
Catboost	0.798
Lasso Logistic Regression	0.795

L. Feature importance



Sources

David, J., Arslan, C. and Robberechts, P. (2020). *Enhancing xG Models with freeze frame data*. [online] DTAI Sports. Available at: https://dtai.cs.kuleuven.be/sports/blog/enhancing-xg-models-with-freeze-frame-data/ [Accessed 10 Dec. 2025].

Davis, J. and Robberechts, P. (2020). Illustrating the interplay between features and models in xG. [online] DTAI Sports. Available at: https://dtai.cs.kuleuven.be/sports/blog/illustrating-the-interplay-between-features-and-models-in-xg/ [Accessed 10 Dec. 2025].

Hewitt, James H., and Oktay Karakuş. “A machine learning approach for player and position adjusted expected goals in football (soccer).” Franklin Open, vol. 4, Aug. 2023, p. 100034, doi:10.1016/j.fraope.2023.100034.

Rathke, A. (2017). An examination of expected goals and shot efficiency in soccer. Journal of Human Sport and Exercise, 12(2proc), S514-S529. doi:https://doi.org/10.14198/jhse.2017.12.Proc2.05